(DEEMED TO BE UNIVERSITY)

Wine Quality Prediction and Feature Importance Analysis Using Random Forest

Dataset: UCI Wine Quality Dataset (Red & White Wine Samples)

Sura Rahul Reddy *2520030305*

Yashwanth *2520030506*

Introduction

- Wine quality is traditionally judged by expert sensory panels, tasting and scoring each sample.
- Sensory evaluation is slow, expensive, and can vary from one taster to another.
- Every wine sample also has measurable physicochemical properties acidity, sugar, sulphates, alcohol, and more.
- These properties can be captured instantly in a lab, unlike a human panel's judgment.
- Our project uses Machine Learning to predict a wine's quality score directly from its physicochemical properties.
- The proposed model is a Random Forest, which also reveals which properties matter most for quality.

Problem Statement

- Assessing wine quality consistently is difficult using traditional approaches such as:
  - Manual sensory tasting panels
  - Subjective scoring that varies between experts
  - Limited ability to scale across large production batches

These approaches make it hard for producers to flag lower-quality batches early or understand exactly which chemical properties drive quality.

Main Goal:

To build a Machine Learning system that predicts wine quality from physicochemical measurements and ranks the features that most influence that prediction, using an Elastic-Net-style Random Forest analysis.

Objectives

- Collect and load the red and white wine physicochemical datasets.
- Combine both datasets into a single, unified dataframe.
- Understand and summarize the structure of the dataset.
- Clean and verify the data before analysis.
- Perform Exploratory Data Analysis (EDA) to uncover patterns related to quality.
- Study feature correlations with wine quality.
- Build and evaluate a Random Forest model, and rank feature importance.

Dataset Overview

Dataset Used: UCI Wine Quality Dataset

- The dataset contains physicochemical measurements of wine samples.
- It includes acidity, sugar, chlorides, sulfur dioxide levels, density, pH, sulphates and alcohol.
- Two files are provided — winequality-red.csv and winequality-white.csv — combined into one dataset of 6,497 rows and 13 columns.
- Our target variable is quality, an integer score.
  - wine_type is added during merging to record Red or White.
- This can be framed as a classification problem (e.g. good vs. poor) or a regression problem predicting the raw score.

Step 1 — Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Library	Purpose
Pandas	Load and manipulate the dataset
NumPy	Numerical calculations
Matplotlib	Create graphs and visualizations
Seaborn	Create statistical visualizations

Step 2 — Load the Dataset

Loading the Dataset

```
from google.colab import files
uploaded = files.upload()

red = pd.read_csv("winequality-red.csv", sep=";")
white = pd.read_csv("winequality-white.csv",
sep=";")
```

Display the Dataset

```
red.head()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	wine_type
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	Red
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5	Red
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	Red
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	Red
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	Red

Check Dataset Size

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

Expected — Rows: 6,497 Columns: 13 (1,599 red + 4,898 white)

Check Column Names

```
print(df.columns.tolist())
```

13 columns: 11 physicochemical features, quality (target) and wine_type.

Step 3 — Combine Datasets & Understand Structure

Tag & Combine Red + White

```
red["wine_type"] = "Red"
white["wine_type"] = "White"
df = pd.concat([red, white], ignore_index=True)
```

1,599 red rows + 4,898 white rows = 6,497 total rows, with a new wine_type column.

Display the Combined Dataset

```
df.head()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	wine_type
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	Red
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5	Red
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	Red
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	Red
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	Red

Check Statistical Summary

```
df.describe()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
count	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000
mean	7.215307	0.339666	0.318633	5.443235	0.056034	30.525319	115.744574	0.994697	3.218501	0.531268	10.491801	5.818378
std	1.296434	0.164636	0.145318	4.757804	0.035034	17.749400	56.521855	0.002999	0.160787	0.148806	1.192712	0.873255
min	3.800000	0.080000	0.000000	0.600000	0.009000	1.000000	6.000000	0.987110	2.720000	0.220000	8.000000	3.000000
25%	6.400000	0.230000	0.250000	1.800000	0.038000	17.000000	77.000000	0.992340	3.110000	0.430000	9.500000	5.000000
50%	7.000000	0.290000	0.310000	3.000000	0.047000	29.000000	118.000000	0.994890	3.210000	0.510000	10.300000	6.000000
75%	7.700000	0.400000	0.390000	8.100000	0.065000	41.000000	156.000000	0.996990	3.320000	0.600000	11.300000	6.000000
max	15.900000	1.580000	1.660000	65.800000	0.611000	289.000000	440.000000	1.038980	4.010000	2.000000	14.900000	9.000000

Step 4 — Missing Values, Duplicates & Target

Check Missing Values & Duplicates

```
df.isnull().sum()
df.duplicated().sum()
```

Total No of missing values:0

```
Duplicate rows: 1177
```

Wine Type Distribution

```
df["wine_type"].value_counts()
```

wine_type	count
White	4898
Red	1599

Understand the Target Variable

```
df["quality"].value_counts().sort_index()
```

```
Wine Quality Distribution:
quality
3      30
4     216
5    2138
6    2836
7    1079
8     193
9       5
Name: count, dtype: int64
```

Step 5 — Data Cleaning

Check Data Types & Re-verify

```
print("Data Types:")
print(df.dtypes)

print("\nTotal Missing Values:")
print(df.isnull().sum().sum())

print("\nTotal Duplicate Rows:")
print(df.duplicated().sum())
```

```
Data Types:
fixed acidity           float64
volatile acidity        float64
citric acid             float64
residual sugar          float64
chlorides               float64
free sulfur dioxide     float64
total sulfur dioxide    float64
density                 float64
pH                      float64
sulphates               float64
alcohol                 float64
quality                   int64
wine_type                object
dtype: object

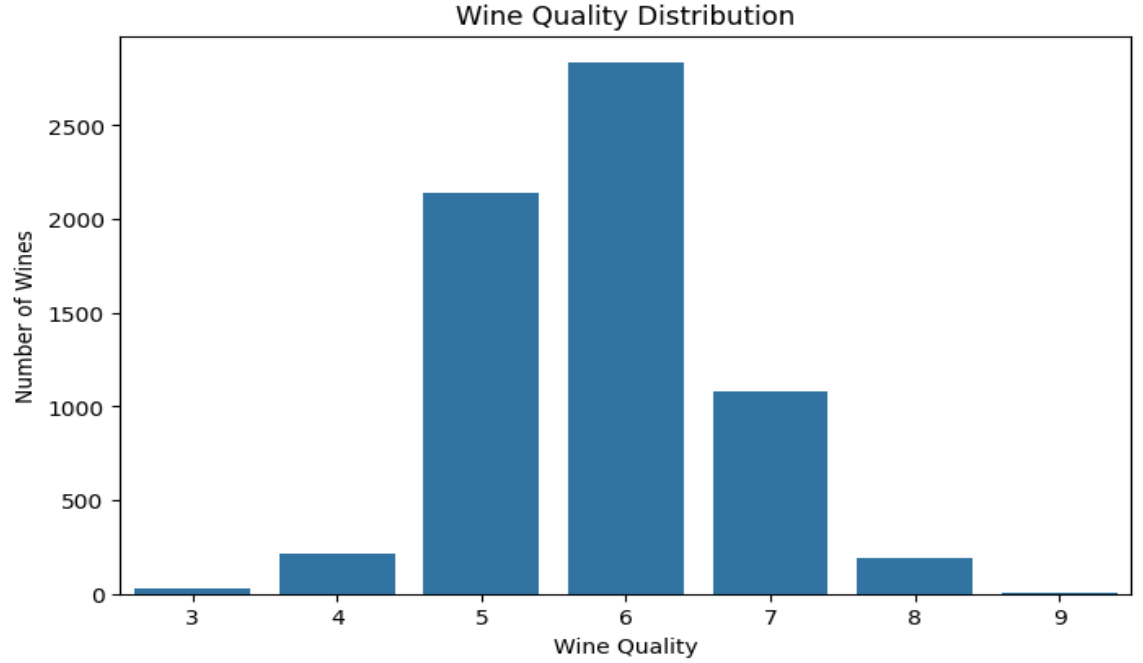
Total Missing Values:
0

Total Duplicate Rows:
1177
```

Step 6 — EDA: Quality & Wine Type Distribution

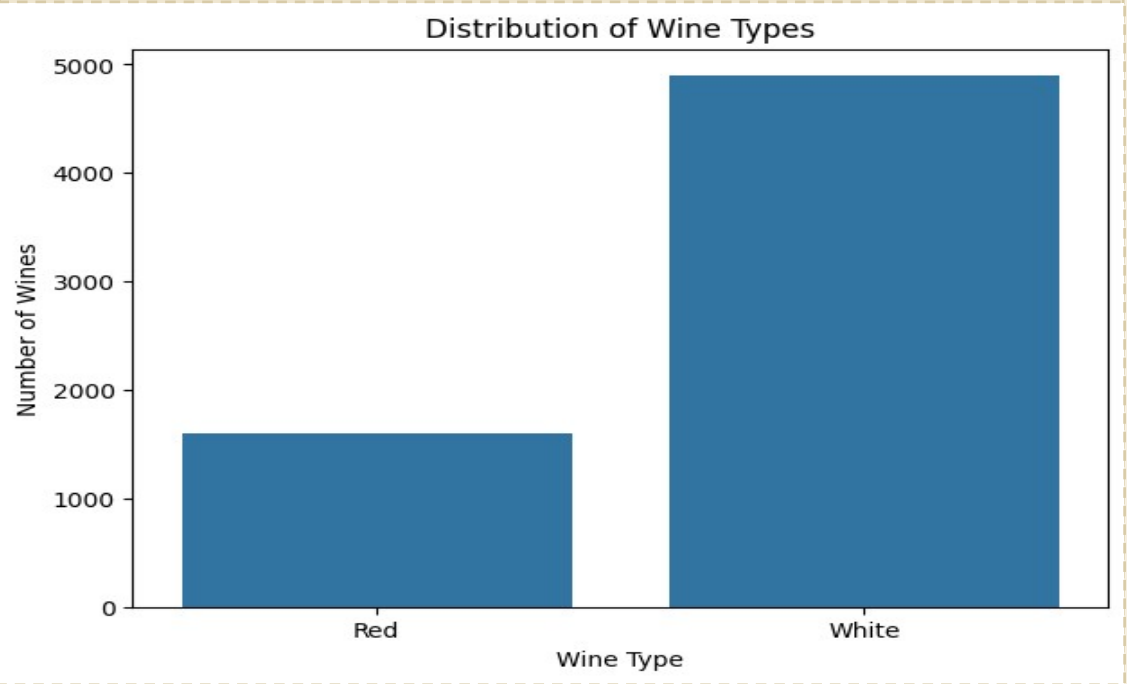
Wine Quality Distribution

```
sns.countplot(data=df, x="quality")
plt.title("Wine Quality Distribution")
plt.show()
```



Red vs White Wine Counts

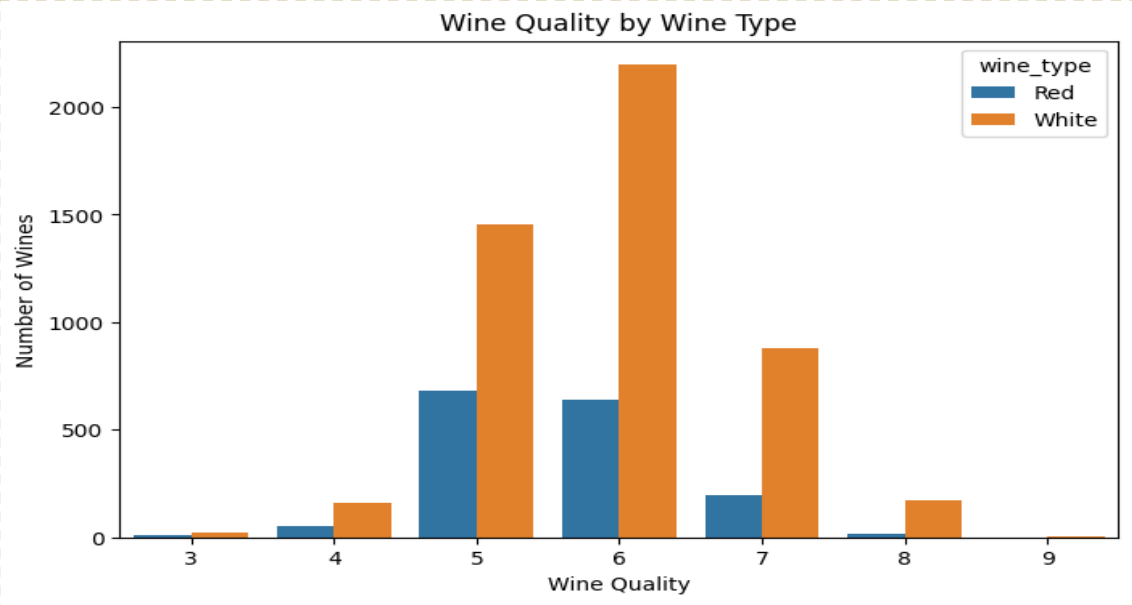
```
sns.countplot(data=df, x="wine_type")
plt.title("Distribution of Wine Types")
plt.show()
```



Step 6 — EDA: Quality by Type & Alcohol

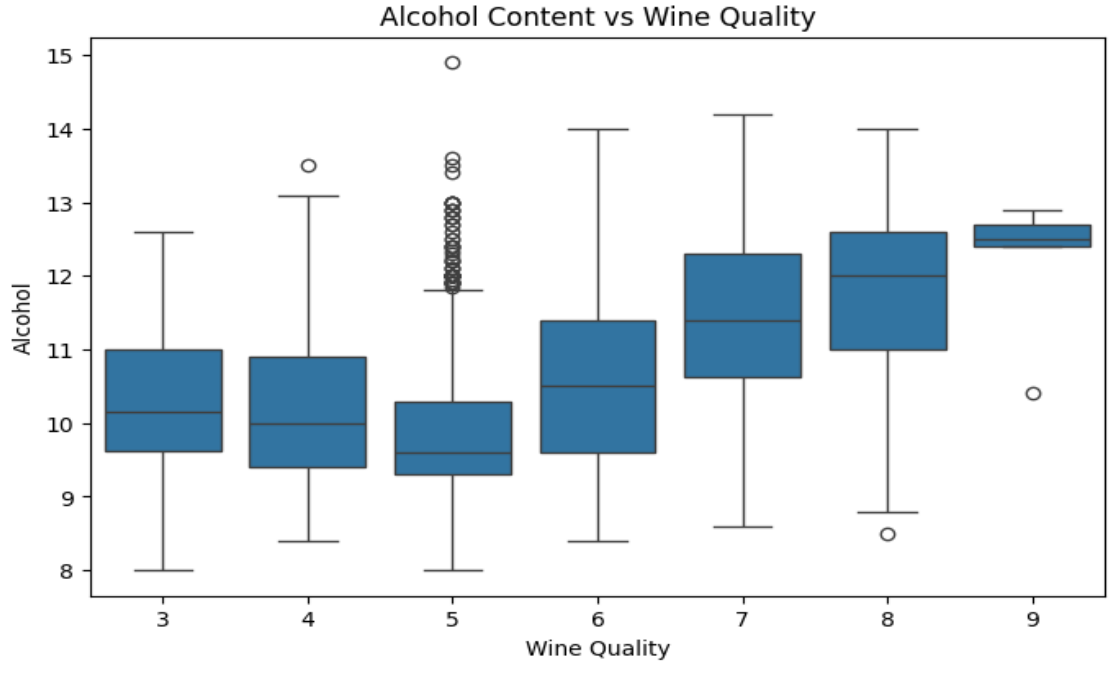
Quality by Wine Type

```
sns.countplot(data=df, x="quality",
    hue="wine_type")
plt.title("Wine Quality by Wine Type")
plt.show()
```



Alcohol vs Quality

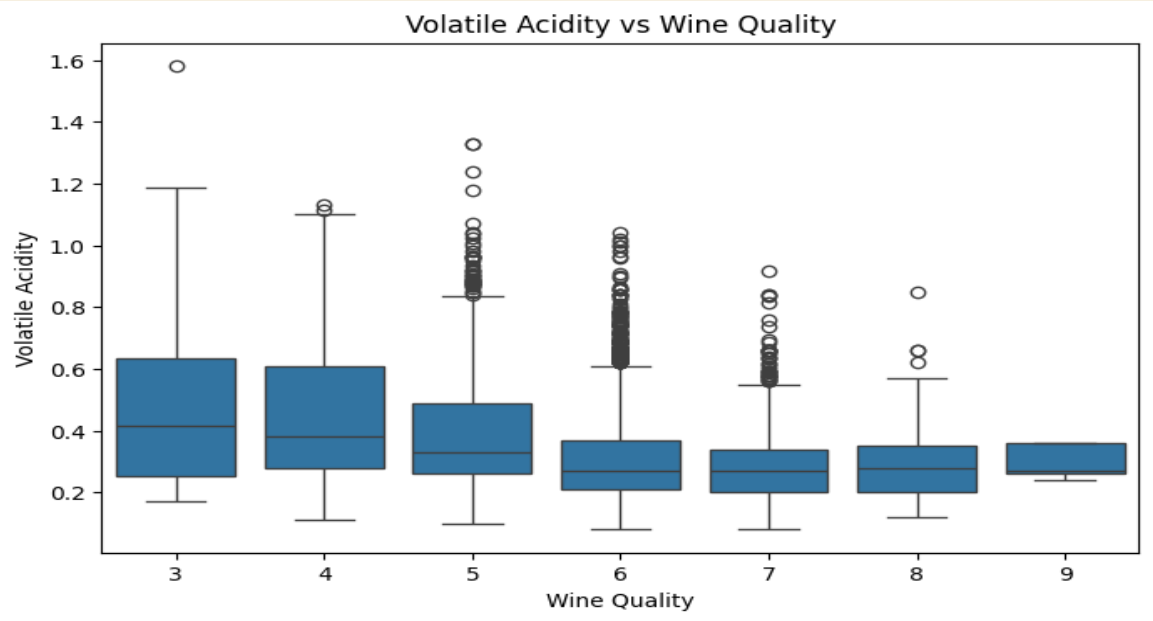
```
sns.boxplot(data=df, x="quality", y="alcohol")
plt.title("Alcohol Content vs Wine Quality")
plt.show()
```



Step 6 — EDA: Volatile Acidity & Correlation Heatmap

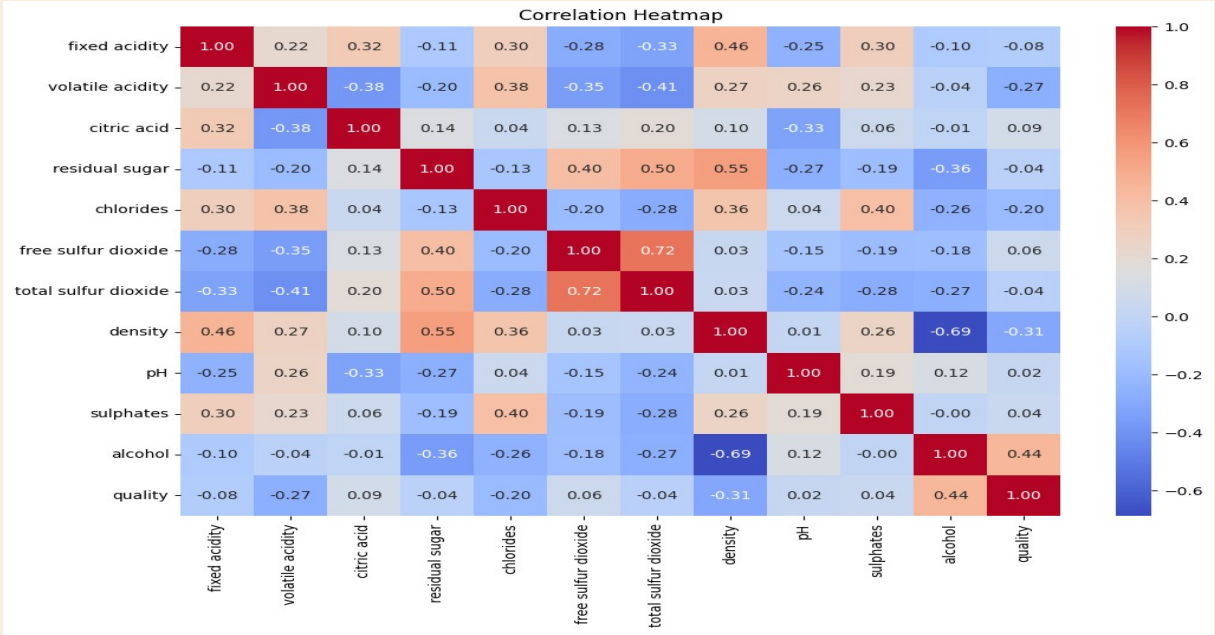
Volatile Acidity vs Quality

```
sns.boxplot(data=df, x="quality",
    y="volatile acidity")
plt.title("Volatile Acidity vs Wine Quality")
plt.show()
```



Correlation Heatmap

```
correlation = df.select_dtypes(
    include=np.number).corr()
sns.heatmap(correlation, annot=True,
    cmap="coolwarm", fmt=".2f")
plt.show()
```



Step 6 — EDA: Correlation With Quality

Correlation With Quality

```
quality_correlation = (
    df.select_dtypes(include=np.number)
      .corr()["quality"]
      .sort_values(ascending=False)
)
 print(quality_correlation)
```

```
quality                1.000000
alcohol                0.444319
citric acid            0.085532
free sulfur dioxide    0.055463
sulphates              0.038485
pH                     0.019506
residual sugar        -0.036980
total sulfur dioxide  -0.041385
fixed acidity         -0.076743
chlorides             -0.200666
volatile acidity      -0.265699
density               -0.305858
Name: quality, dtype: float64
```

Step 6 — EDA: Correlation With Quality

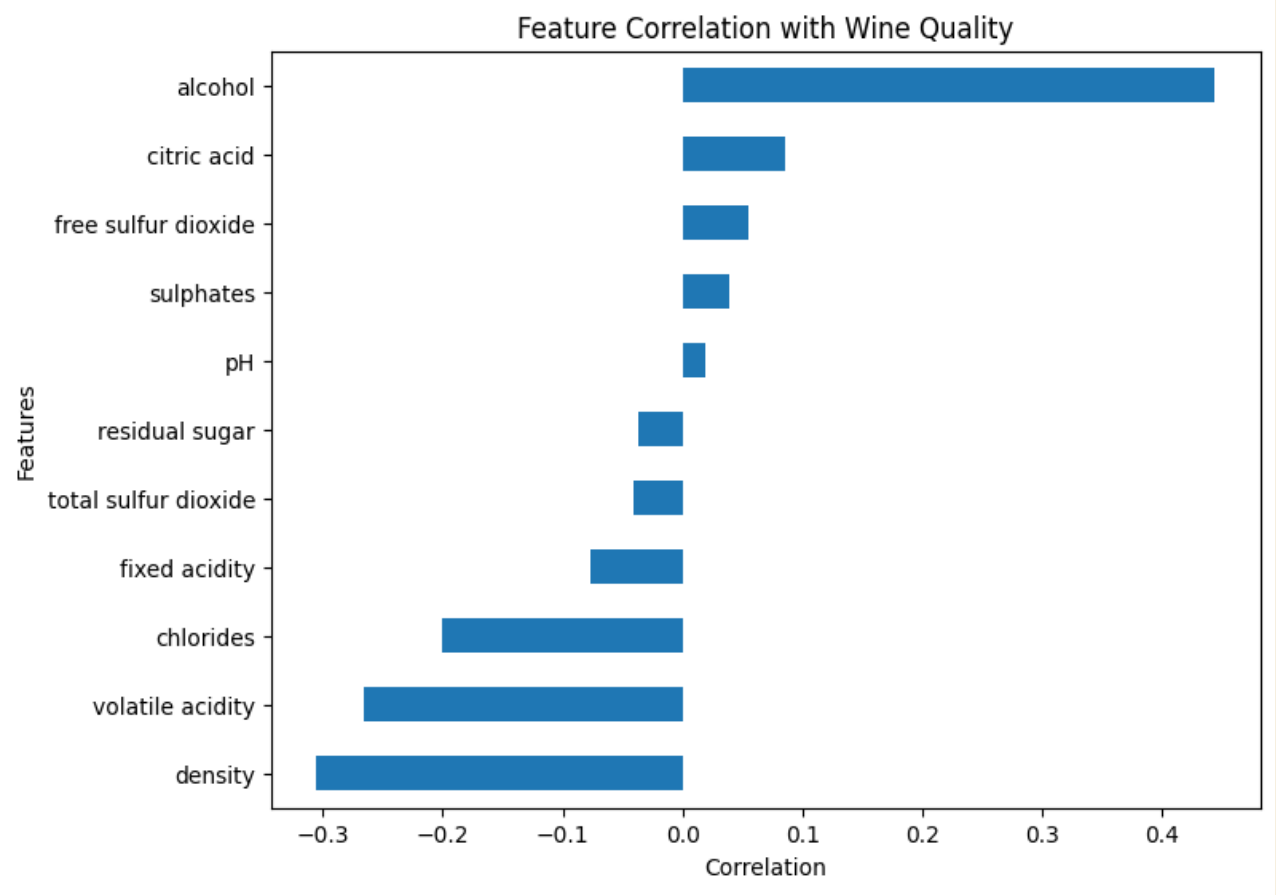
Correlation With Quality

```
plt.figure(figsize=(8,6))

quality_correlation.drop("quality").sort_values().plot
(
    kind="barh"
)

plt.title("Feature Correlation with Wine Quality")
plt.xlabel("Correlation")
plt.ylabel("Features")

plt.show()
```



Step 7 — Data Cleaning

Code

```
print("Missing values per column:")
print(df.isnull().sum())
print("Total missing values:", df.isnull().sum().sum())
print("Duplicate rows before removal:", df.duplicated().sum())
df_clean = df.dropna()
df_clean = df_clean.drop_duplicates().reset_index(drop=True)
print("Duplicate rows after removal:", df_clean.duplicated().sum())
print("Missing values after cleaning:",
    df_clean.isnull().sum().sum())
print("Shape before:", df.shape)
print("Shape after:", df_clean.shape)
print(df_clean.dtypes)
print(df_clean.head())
quality_counts = df_clean["quality"].value_counts().sort_index()
wine_counts = df_clean["wine_type"].value_counts()
print("Quality distribution:")
print(quality_counts)
print("Wine type distribution:")
print(wine_counts)
summary = df_clean.describe().round(2)
print(summary)
print(df_clean["quality"].min(), df_clean["quality"].max())
print("Wine type proportions:")
print(df_clean["wine_type"].value_counts(normalize=True).round(3))
```

Cleaning Output

```
Missing values per column:
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
wine_type               0
dtype: int64
Total missing values: 0

Duplicate rows before removal: 1177
Duplicate rows after removal: 0
Missing values after cleaning: 0
Shape before: (6497, 13)
Shape after:  (5320, 13)
```

Step 8 — Feature Preparation

Code

```
SEED = 42
GOOD_THRESHOLD = 6
data = df_clean.copy()
data["is_white"] = (data["wine_type"] == "White").astype(int)
data["good_quality"] = (data["quality"] >=
    GOOD_THRESHOLD).astype(int)
FEATURES = ["fixed acidity", "volatile acidity", "citric acid",
            "residual sugar", "chlorides", "free sulfur dioxide",
            "total sulfur dioxide", "density", "pH", "sulphates",
            "alcohol", "is_white"]
X = data[FEATURES]
y = data["good_quality"]
print("Features:", X.shape, "Target:", y.shape)
print("Class balance (%):")
print((y.value_counts(normalize=True).sort_index() * 100).round(2))
print(data[FEATURES].head())
print("Target distribution:")
print(y.value_counts().sort_index())
print("Target proportions:")
print((y.value_counts(normalize=True) * 100).round(2))
print("Number of input features:", len(FEATURES))
print("Good wine samples:", int(y.sum()))
print("Poor wine samples:", int((y == 0).sum()))
print(data[["quality", "good_quality", "wine_type"]].head())
```

Feature / Target Summary

```
Features: (5320, 12)  Target: (5320,)

Class balance (%):
good_quality
Poor    37.37
Good    62.63
Name: proportion, dtype: float64
```

Step 9 — Train/Test Split & Baseline Model

Code

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED)
print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
rf = RandomForestClassifier(n_estimators=200, random_state=SEED,
    n_jobs=-1)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
print("Baseline Random Forest")
print("Train accuracy:", round(rf.score(X_train, y_train), 4))
print("Test accuracy:", round(accuracy_score(y_test, y_pred), 4))
print(classification_report(y_test, y_pred,
                            target_names=["Poor", "Good"], digits=3))
print("Training labels:", y_train.value_counts().to_dict())
print("Testing labels:", y_test.value_counts().to_dict())
cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix:")
print(cm)
print("Training rows:", len(X_train))
print("Testing rows:", len(X_test))
print("Actual test labels:", y_test.value_counts().to_dict())
print("Predicted labels:", pd.Series(y_pred).value_counts().to_dict())
print("Correct predictions:", int((y_pred == y_test).sum()))
print("Incorrect predictions:", int((y_pred != y_test).sum()))
```

Baseline Classification Results

```
Baseline Random Forest
Train accuracy: 1.0
Test accuracy:  0.7716

              precision    recall  f1-score   support

        Poor      0.717     0.643     0.678       398
        Good      0.799     0.848     0.823       666

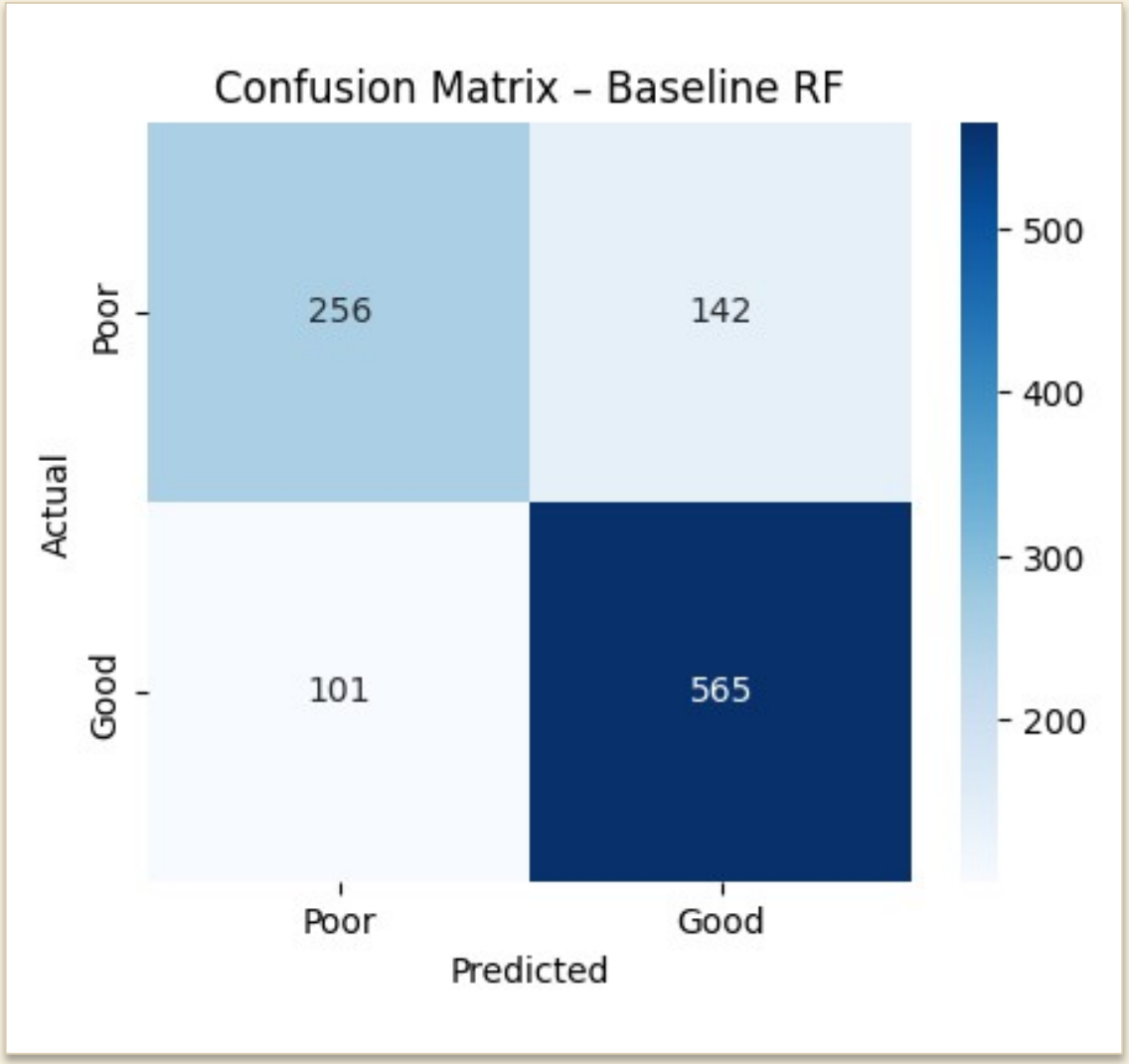
    accuracy                          0.772      1064
   macro avg      0.758     0.746     0.751      1064
weighted avg      0.768     0.772     0.769      1064
```

Step 10 — Baseline Confusion Matrix

Code

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Poor", "Good"],
            yticklabels=["Poor", "Good"])
plt.title("Confusion Matrix – Baseline RF")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()
print(cm)
print("Total test rows:", cm.sum())
print("Correct predictions:", cm.trace())
print("Incorrect predictions:", cm.sum() - cm.trace())
tn, fp, fn, tp = cm.ravel()
print("True negatives:", tn)
print("False positives:", fp)
print("False negatives:", fn)
print("True positives:", tp)
plt.xticks(rotation=0)
plt.yticks(rotation=0)
print("Matrix shape:", cm.shape)
print("Matrix total:", int(cm.sum()))
print("Accuracy from matrix:", round(cm.trace() / cm.sum(), 4))
```

Confusion Matrix Output



Step 11 — Cross-Validation

Code

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
cv_acc = cross_val_score(rf, X_train, y_train, cv=cv,
                         scoring="accuracy", n_jobs=-1)
cv_f1 = cross_val_score(rf, X_train, y_train, cv=cv,
                        scoring="f1", n_jobs=-1)
print("CV accuracy per fold:", cv_acc.round(4))
print("Mean CV accuracy:", round(cv_acc.mean(), 4))
print("Accuracy standard deviation:", round(cv_acc.std(), 4))
print("Mean CV F1:", round(cv_f1.mean(), 4))
print("F1 standard deviation:", round(cv_f1.std(), 4))
cv_summary = pd.DataFrame({"Accuracy": cv_acc, "F1": cv_f1})
cv_summary.index = range(1, len(cv_summary) + 1)
print(cv_summary.round(4))
print("Accuracy range:", round(cv_acc.min(), 4), round(cv_acc.max(),
4))
print("F1 range:", round(cv_f1.min(), 4), round(cv_f1.max(), 4))
print("Fold count:", len(cv_summary))
cv_summary = pd.DataFrame({"Accuracy": cv_acc, "F1": cv_f1})
cv_summary.index = range(1, len(cv_summary) + 1)
print(cv_summary.to_string(index=False))
print("Best fold accuracy:", round(cv_acc.max(), 4))
print("Lowest fold accuracy:", round(cv_acc.min(), 4))
print("Best fold F1:", round(cv_f1.max(), 4))
print("Lowest fold F1:", round(cv_f1.min(), 4))
print("Accuracy standard error:", round(cv_acc.std() / len(cv_acc) **
0.5, 4))
```

Cross-Validation Results

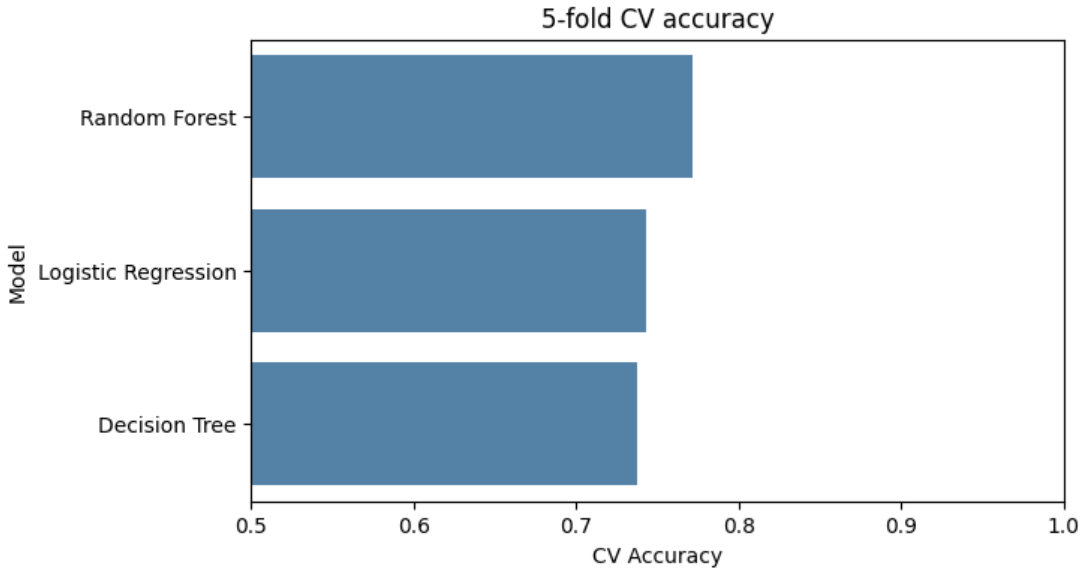
```
CV accuracy per fold: [0.784  0.7897 0.7685 0.765  0.7532]
Mean CV accuracy: 0.7721 (+/- 0.0132)
Mean CV F1:       0.8223 (+/- 0.0115)
```

Step 12 — Model Comparison

Code

```
candidates = {
    "Logistic Regression": make_pipeline(StandardScaler(),
                                         LogisticRegression(max_iter=
            2000)),
    "Decision Tree": DecisionTreeClassifier(max_depth=6,
        random_state=SEED),
    "Random Forest": RandomForestClassifier(n_estimators=200,
                                            random_state=SEED,
            n_jobs=-1)
}
rows = []
for name, model in candidates.items():
    scores = cross_val_score(model, X_train, y_train, cv=cv,
                             scoring="accuracy", n_jobs=-1)
    rows.append((name, scores.mean(), scores.std()))
cmp_df = pd.DataFrame(rows, columns=["Model", "CV Accuracy", "Std"])
cmp_df = cmp_df.sort_values("CV Accuracy", ascending=False)
display(cmp_df.round(4))
sns.barplot(data=cmp_df, x="CV Accuracy", y="Model",
    color="steelblue")
plt.xlim(0.5, 1)
plt.title("5-fold CV accuracy")
plt.show()
```

Model Comparison Chart



Step 13 — Hyperparameter Tuning

Code

```
param_dist = {
    "n_estimators": [200, 300, 500, 800],
    "max_depth": [None, 10, 15, 20, 30],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2", 0.5]
}
search = RandomizedSearchCV(
    RandomForestClassifier(random_state=SEED, n_jobs=-1),
    param_dist, n_iter=25, scoring="f1", cv=cv,
    random_state=SEED, n_jobs=-1)
search.fit(X_train, y_train)
print("Best CV F1:", round(search.best_score_, 4))
print("Best params:", search.best_params_)
best_rf = search.best_estimator_
results = pd.DataFrame(search.cv_results_)
results = results.sort_values("mean_test_score", ascending=False)
print(results[["mean_test_score", "params"]].head(3))
print("Selected model:", type(best_rf).__name__)
best_params = search.best_params_
print("Tree count:", best_params["n_estimators"])
print("Maximum depth:", best_params["max_depth"])
print("Minimum split size:", best_params["min_samples_split"])
print("Feature sampling:", best_params["max_features"])
```

Best Parameters

```
Best CV F1: 0.822
Best params: {'n_estimators': 300, 'min_samples_split': 2,
'min_samples_leaf': 1, 'max_features': 'log2', 'max_depth': 30}
```

Step 14 — Tuned Model Metrics

Code

```
y_pred = best_rf.predict(X_test)
y_prob = best_rf.predict_proba(X_test)[:, 1]
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred), 4))
print("Recall:", round(recall_score(y_test, y_pred), 4))
print("F1-score:", round(f1_score(y_test, y_pred), 4))
print("ROC-AUC:", round(roc_auc_score(y_test, y_prob), 4))
print(classification_report(y_test, y_pred,
                            target_names=["Poor", "Good"], digits=3))
cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix:")
print(cm)
print("Correct predictions:", int(cm.trace()))
print("Incorrect predictions:", int(cm.sum() - cm.trace()))
print("Test samples:", len(y_test))
print("Predicted labels:", pd.Series(y_pred).value_counts().to_dict())
report = classification_report(y_test, y_pred,
                               target_names=["Poor", "Good"],
                               output_dict=True)
print("Poor class recall:", round(report["Poor"]["recall"], 4))
print("Good class recall:", round(report["Good"]["recall"], 4))
```

Final Classification Metrics

```
Accuracy : 0.7716
Precision: 0.8000
Recall   : 0.8468
F1-score : 0.8228
ROC-AUC  : 0.8375

              precision    recall  f1-score   support

        Poor      0.716     0.646     0.679       398
        Good      0.800     0.847     0.823       666

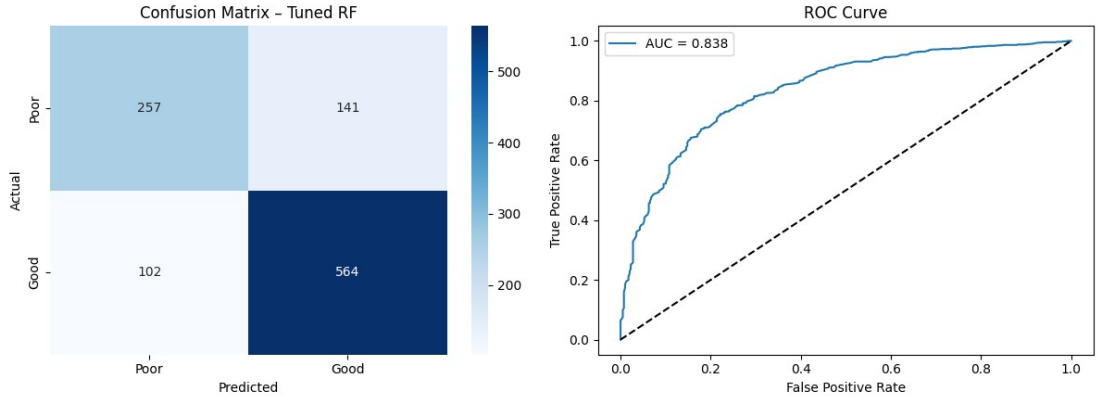
    accuracy                          0.772      1064
   macro avg      0.758     0.746     0.751      1064
weighted avg      0.769     0.772     0.769      1064
```

Step 15 — Confusion Matrix & ROC Curve

Code

```
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Poor", "Good"],
            yticklabels=["Poor", "Good"], ax=ax[0])
ax[0].set_title("Confusion Matrix – Tuned RF")
ax[0].set_xlabel("Predicted")
ax[0].set_ylabel("Actual")
fpr, tpr, _ = roc_curve(y_test, y_prob)
ax[1].plot(fpr, tpr, label=f"AUC = {roc_auc_score(y_test,
    y_prob):.3f}")
ax[1].plot([0, 1], [0, 1], "k--")
ax[1].set_title("ROC Curve")
ax[1].set_xlabel("False Positive Rate")
ax[1].set_ylabel("True Positive Rate")
ax[1].legend()
plt.tight_layout()
plt.show()
print("Confusion matrix:")
print(cm)
print("True negatives:", cm[0, 0])
print("False positives:", cm[0, 1])
print("False negatives:", cm[1, 0])
print("True positives:", cm[1, 1])
print("ROC-AUC:", round(roc_auc_score(y_test, y_prob), 4))
```

Confusion Matrix & ROC Output

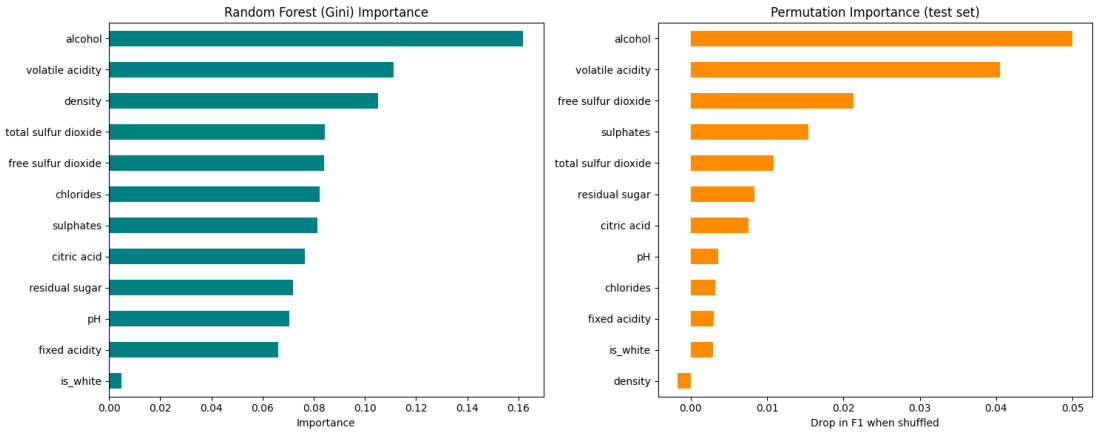


Step 16 — Feature Importance Analysis

Code

```
gini = pd.Series(best_rf.feature_importances_,
    index=FEATURES).sort_values()
perm = permutation_importance(best_rf, X_test, y_test,
                              scoring="f1", n_repeats=20,
                              random_state=SEED, n_jobs=-1)
perm_s = pd.Series(perm.importances_mean,
    index=FEATURES).sort_values()
fig, ax = plt.subplots(1, 2, figsize=(15, 6))
gini.plot(kind="barh", color="teal", ax=ax[0])
ax[0].set_title("Random Forest (Gini) Importance")
ax[0].set_xlabel("Importance")
perm_s.plot(kind="barh", color="darkorange", ax=ax[1])
ax[1].set_title("Permutation Importance (test set)")
ax[1].set_xlabel("Drop in F1 when shuffled")
plt.tight_layout()
plt.show()
rank = pd.DataFrame({"Gini importance": gini, "Permutation
    importance": perm_s})
display(rank.sort_values("Gini importance",
    ascending=False).round(4))
top_gini = gini.sort_values(ascending=False).head(5)
top_perm = perm_s.sort_values(ascending=False).head(5)
print("Top Gini features:")
print(top_gini)
print("Top permutation features:")
print(top_perm)
```

Feature Importance Charts

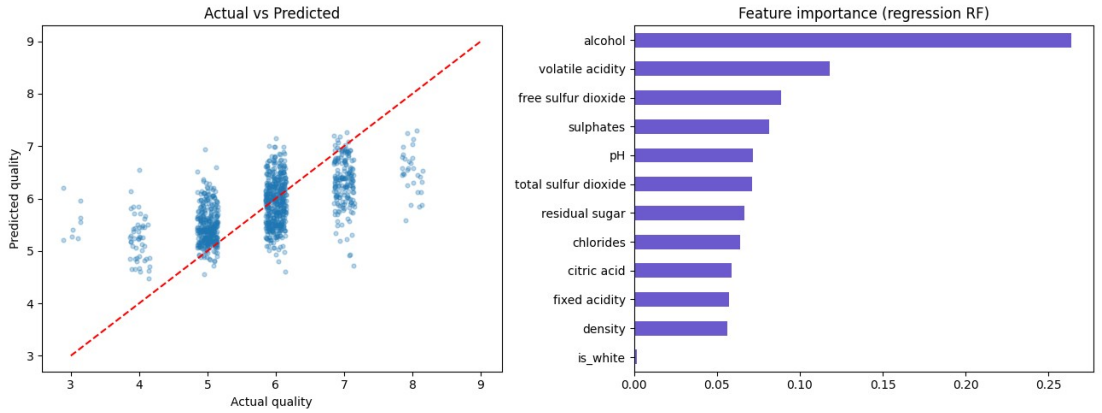


Step 17 — Regression Analysis

Code

```
y_reg = data["quality"]
reg = RandomForestRegressor(n_estimators=500, random_state=SEED,
    n_jobs=-1)
reg.fit(X_train, y_reg.loc[X_train.index])
pred = reg.predict(X_test)
actual = y_reg.loc[X_test.index]
print("RMSE:", round(np.sqrt(mean_squared_error(actual, pred)), 3))
print("MAE:", round(mean_absolute_error(actual, pred), 3))
print("R²:", round(r2_score(actual, pred), 3))
fig, ax = plt.subplots(1, 2, figsize=(13, 5))
ax[0].scatter(actual, pred, alpha=0.3, s=12)
ax[0].plot([3, 9], [3, 9], "r--")
ax[0].set_xlabel("Actual quality")
ax[0].set_ylabel("Predicted quality")
ax[0].set_title("Actual vs Predicted")
pd.Series(reg.feature_importances_,
    index=FEATURES).sort_values().plot(kind="barh", ax=ax[1])
ax[1].set_title("Feature importance (regression RF)")
plt.tight_layout()
plt.show()
error = actual - pred
print("Mean residual:", round(error.mean(), 3))
print("Median residual:", round(error.median(), 3))
pred_table = pd.DataFrame([actual, pred, error]).T
print(pred_table.head())
```

Regression Results



Step 18 — Save Model & Predict New Wine

Code

```
joblib.dump({"model": best_rf, "features": FEATURES,
             "threshold": GOOD_THRESHOLD}, "wine_rf_model.joblib")
print("Saved -> wine_rf_model.joblib")
new_wine = pd.DataFrame([{
    "fixed acidity": 7.4, "volatile acidity": 0.70,
    "citric acid": 0.0, "residual sugar": 1.9,
    "chlorides": 0.076, "free sulfur dioxide": 11,
    "total sulfur dioxide": 34, "density": 0.9978,
    "pH": 3.51, "sulphates": 0.56,
    "alcohol": 9.4, "is_white": 0
}])[FEATURES]
prob = best_rf.predict_proba(new_wine)[0, 1]
label = "GOOD" if prob >= 0.5 else "POOR"
print("Predicted:", label)
print("P(good):", round(prob, 2))
print("Input feature count:", len(FEATURES))
print("Probability for good:", round(prob, 4))
print("Probability threshold:", 0.5)
print("Predicted quality class:", label)
print("Probability threshold:", 0.5)
print(new_wine[FEATURES].T)
```

Sample Prediction Output

```
Saved -> wine_rf_model.joblib
Predicted: POOR  (P(good) = 0.04)
```

Thank You